НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №4-5. AI-агент для Web Quality Audit: accessibility, технічне SEO та Core Web Vitals

з дисципліни «Сучасна веброзробка: HTML, CSS, JS/TS»

Термін здачі: 30.09.2026

1. Мета роботи

Дослідити основні підходи до комплексного аудиту якості вебсторінки: доступності (accessibility), технічного SEO та продуктивності. Опанувати Chrome DevTools, Lighthouse та автоматизовані accessibility-перевірки для виявлення проблем. Навчитися використовувати AI-агента як інструмент аналізу, класифікації та виправлення проблем вебсторінки, зберігаючи контроль над правильністю запропонованих рішень.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Розуміти основні принципи Web Accessibility та структуру WCAG 2.2, зокрема те, які критерії з'явилися саме у версії 2.2.
- Уміти перевіряти вебсторінку клавіатурою, через Chrome DevTools та Accessibility Tree.
- Розуміти призначення ARIA та випадки, коли її використання є виправданим.
- Уміти застосовувати автоматизовані accessibility-перевірки та пояснювати їхні обмеження.
- Уміти проводити базовий технічний SEO-аудит вебсторінки.
- Розуміти призначення JSON-LD / Schema.org, Open Graph, `robots.txt` та `sitemap.xml`, а також чому місце розміщення `robots.txt` має значення.
- Розуміти відмінність між crawling та indexing.
- Розуміти призначення та основні метрики Core Web Vitals (LCP, INP, CLS) і відмінність між lab-даними та польовими даними.

- Уміти використовувати Lighthouse та Chrome DevTools для аналізу performance і ставити коректні умови вимірювання.
- Уміти використовувати AI-агента для комплексного аудиту вебсторінки.
- Уміти критично перевіряти рекомендації AI за документацією та результатами автоматизованих і ручних перевірок.
- Уміти виконувати цикл **audit** → **verification** → **fix** → **re-audit**.

3. Завдання лабораторної роботи

1. Провести серію досліджень з accessibility, автоматизованого тестування, технічного SEO та Core Web Vitals, спираючись на офіційну документацію [1]–[15] та експерименти в браузері.
2. Провести первинний аудит наданого starter-проєкту за допомогою DevTools та автоматизованих інструментів.
3. Виконати AI-assisted комплексний аудит і сформувати перелік виявлених проблем та рекомендацій.
4. Перевірити рекомендації AI за документацією та результатами власних експериментів.
5. Виправити обґрунтовані проблеми та повторно провести аудит.
6. Оформити звіт-дослідження, який демонструє не лише фінальний результат, а й процес **виявлення** → **перевірки** → **виправлення** → **повторної перевірки**.
7. Усно захистити роботу.

> **Важливо!** Практична частина виконується на основі наданого starter-проєкту, який містить навмисно допущені проблеми різного типу. Завдання студента — самостійно виявити, **підтвердити** та виправити їх, а не відтворити готове рішення.

4. Дослідницькі завдання

Перед виконанням практичної частини опрацюйте теоретичні матеріали та проведіть експерименти. Результати (спостереження та висновки) додайте до звіту. Усі рішення перевіряйте за джерелами [1]–[15].

4.1. Web Accessibility та WCAG 2.2

Опрацювати:

1. Web Content Accessibility Guidelines (WCAG) 2.2 — W3C
2. WCAG 2 at a Glance — W3C
3. What's New in WCAG 2.2 — W3C WAI

Дослідити чотири принципи WCAG (Perceivable, Operable, Understandable, Robust), поняття Success Criterion, рівні відповідності A / AA / AAA, а також keyboard accessibility, focus, accessible names, contrast, форми й labels, semantic HTML.

Окремо визначте, **що з'явилося саме у WCAG 2.2**: 2.4.11 Focus Not Obscured (Minimum), 2.5.7 Dragging Movements, 2.5.8 Target Size (Minimum), 3.2.6 Consistent Help, 3.3.7 Redundant Entry, 3.3.8 Accessible Authentication (Minimum) — і те, що критерій 4.1.1 Parsing у версії 2.2 визнано застарілим і вилучено.

Для контрасту зафіксуйте конкретні порогові значення, бо на захисті знадобиться не оцінка «виглядає бліденько», а число:

Критерій	Що вимірює	Мінімум для AA
1.4.3 Contrast (Minimum)	звичайний текст	4,5 : 1
1.4.3 Contrast (Minimum)	великий текст (≥ 24px або ≥ 18,66px bold)	3 : 1
1.4.11 Non-text Contrast	межі полів, іконки, стани UI	3 : 1
2.5.8 Target Size	розмір цілі для вказівника	24 × 24 CSS px

Експеримент

Виберіть один інтерактивний елемент сторінки та перевірте його мишею, клавіатурою і через DevTools → Accessibility Tree. Порівняйте результати:

- чи можна дістатися елемента клавіатурою;
- чи має він доступну назву;
- яку роль бачить браузер;
- чи відповідає accessibility-представлення фактичному призначенню елемента.

4.2. Accessibility у Chrome DevTools

Опрацювати:

4. Accessibility features reference — Chrome DevTools

Дослідити: Accessibility Tree, ARIA attributes, Computed Accessibility Properties, Source Order Viewer, перевірку контрастності.

Accessibility Tree є підмножиною DOM Tree і містить елементи, релевантні для представлення вмісту допоміжним технологіям.

Експеримент

Для **двох** елементів різного типу (наприклад заголовка та інтерактивного елемента) відкрийте вкладку **Accessibility** і визначте Role, Name, доступні властивості та положення в дереві.

Окремо перевірте порядок переходу сторінкою клавішею Tab і зробіть висновок, чи відповідає порядок фокусування логіці сторінки. Якщо порядок відрізняється від візуального — скористайтеся **Source Order Viewer**, щоб побачити причину.

4.3. Автоматизований accessibility audit

Опрацювати:

5. Lighthouse — Chrome for Developers
6. axe-core — Accessibility engine

Дослідити:

- які accessibility-проблеми можна виявити автоматично;
- що означають результати `violations`, `passes` та `incomplete`;
- чому автоматизований аудит не може повністю визначити accessibility вебсторінки;
- які проблеми потрібно перевіряти вручну.

Два факти, важливі для висновків цієї роботи. По-перше, категорія Accessibility у Lighthouse побудована на axe-core, тому Lighthouse і розширення axe DevTools у значній частині правил дадуть той самий результат — це не дві незалежні перевірки. По-друге, набір автоматизованих правил покриває переважно критерії WCAG 2.0 / 2.1 рівнів A та AA; більшість нових критеріїв 2.2 з §4.1 автоматично не перевіряється взагалі.

> Перевірка за допомогою скрінрідера (NVDA, VoiceOver) виходить за межі цієї роботи. Ручна перевірка тут означає клавіатуру та Accessibility Tree.

Експеримент

Запустіть accessibility audit для starter-проєкту та зафіксуйте: знайдені проблеми, елементи, яких вони стосуються, тип / рівень проблеми, рекомендацію інструмента.

Для щонайменше двох проблем виконайте ручну перевірку в DevTools і визначте, чи справді проблема присутня. Окремо знайдіть на сторінці **щонайменше одну проблему, якої автоматизований інструмент не показав**.

Висновок: які проблеми можна надійно автоматизувати, а які потребують участі розробника?

4.4. AI-assisted accessibility audit

AI-агенту можна передати HTML, CSS, JavaScript, результати Lighthouse, результати axe та скріншоти окремих ділянок сторінки.

Приклад запиту:

```
Проаналізуй цю вебсторінку з точки зору WCAG 2.2. Не переписуй код одразу.
Спочатку сформуй список потенційних проблем accessibility. Для кожної
вкажи:
1) елемент; 2) проблему; 3) відповідний принцип або Success Criterion WCAG;
4) спосіб перевірки; 5) рекомендований спосіб виправлення.
Окремо познач проблеми, які можна перевірити автоматично, та проблеми, які
потребують ручної перевірки.
```

Дослідити:

1. Чи знаходить AI проблеми, яких не виявив Lighthouse або axe?
2. Чи правильно AI визначає відповідні критерії WCAG?
3. Чи пропонує AI зайві ARIA-атрибути?
4. Чи можуть рекомендації AI суперечити нативній HTML-семантиці?
5. Які рекомендації необхідно перевірити за документацією перед внесенням змін?

> **Головне правило:** AI-рекомендація не є доказом наявності проблеми.

Типові приклади, на яких це правило перевіряється: порада «додайте `aria-label` до всіх кнопок» — для кнопки зі звичайним видимим текстом такий атрибут непотрібний і лише перезаписує доступну назву; порада «додайте `role="button"` до цього `<div>`» — питання в тому, чому замість цього не використати нативний `<button>`, який дає роль, фокус і обробку клавіатури без жодного атрибута.

4.5. Technical SEO audit та structured data

Опрацювати:

7. Introduction to structured data markup in Google Search — Google Search Central
8. Schema Markup Validator — schema.org
9. Rich Results Test — Google

Дослідити: structured data, Schema.org, JSON-LD, `@context`, `@type`, властивості structured data. Google рекомендує саме JSON-LD, якщо такий формат відповідає можливостям проєкту.

Для перевірки розмітки використовуйте обидва інструменти та зверніть увагу на різницю між ними: Schema Markup Validator [8] перевіряє синтаксис і відповідність словника Schema.org для будь-якого типу, а Rich Results Test [9] показує лише типи, для яких Google формує rich results. Валідний JSON-LD, який не дає rich results, — не помилка.

Експеримент

Проаналізуйте starter-проєкт і визначте: чи доцільно використовувати structured data; який тип Schema.org відповідає змісту сторінки; які властивості можна описати; які

дані можна взяти безпосередньо з контенту сторінки.

Окремо перевірте, **чи відповідає наявна на сторінці розмітка тому, що насправді написано в контенті**.

Висновок: чому structured data повинні описувати фактичний зміст сторінки?

4.6. Open Graph, robots.txt та sitemap.xml

Опрацювати:

10. The Open Graph protocol
11. Introduction to robots.txt — Google Search Central
12. Build and Submit a Sitemap — Google Search Central

Open Graph. Дослідіть призначення основних властивостей:

```
<meta property="og:title" content="...">
<meta property="og:description" content="...">
<meta property="og:type" content="website">
<meta property="og:url" content="...">
<meta property="og:image" content="...">
```

Open Graph представляє вебсторінку як об'єкт у social graph. Поясніть, що визначає `og:title`, яку роль виконує `og:description`, для чого потрібен `og:image` і чому `og:url` повинен відповідати фактичній URL сторінки.

robots.txt. Дослідіть директиви `User-agent`, `Allow`, `Disallow`, `Sitemap`, а також відмінність між crawling та indexing. Google визначає `robots.txt` як механізм керування тим, які URL crawler може запитувати; це **не** механізм видалення сторінки з Google Search — для цього існує `noindex`.

sitemap.xml. Дослідіть `<urlset>`, `<url>`, `<loc>`, `<lastmod>`, вимоги до URL (абсолютні, того самого origin) та зв'язок sitemap із `robots.txt`.

Експеримент

Створіть для starter-проєкту `robots.txt` і `sitemap.xml`, після чого перевірте: чи розміщені файли у правильному місці; чи містять коректні дані; чи не блокує `robots.txt` потрібні ресурси або сторінки; чи містить sitemap лише релевантні URL.

> **Де саме перевіряти.** `robots.txt` враховується пошуковими системами лише в корені origin — `https://example.com/robots.txt`. GitHub Pages для project page віддає роботу за шляхом `https://<username>.github.io/<repository>/`, тому файл `.../<repository>/robots.txt` **не є** `robots.txt` цього origin: він залишається звичайним статичним файлом, і crawler його не читає. Vercel дає власний origin, у корені якого `robots.txt` працює як належить, — тому перевірку §4.6 і §5.1 виконуйте на Vercel-деплої. Це та сама причина, через яку `robots.txt` не прибирає сторінку з індексу: обидва питання — про те, що саме і **де саме** читає crawler.

4.7. Core Web Vitals та performance audit

Опрацювати:

13. Web Vitals — web.dev
14. Core Web Vitals workflows with Google tools — web.dev
15. Interaction to Next Paint (INP) — web.dev

Поточний набір Core Web Vitals включає три метрики. Рекомендовані пороги оцінюються для **75-го перцентиля польових даних**:

Метрика	Що вимірює	Рекомендований поріг
LCP	завантаження основного контенту	≤ 2,5 с
INP	швидкість реакції на взаємодію	≤ 200 мс
CLS	візуальну стабільність	≤ 0,1

> **INP не вимірюється стандартним запуском Lighthouse.** Це польова метрика: вона обчислюється з реальних взаємодій користувачів, тому звичайний (navigation) аудит, який лише завантажує сторінку, її не повертає, а для нового навчального сайту не буде й даних CrUX. Режим Lighthouse **Timespan** покаже INP для взаємодій, які ви виконаєте під час запису, — але це одна ваша взаємодія, а не 75-й перцентиль польових даних. **TBT** — lab-проксі лише для блокування потоку під час завантаження: повільний обробник кліку він не побачить. Фактичну затримку взаємодії міряйте у DevTools → **Performance**: запишіть трейс, виконайте взаємодію та знайдіть довгу задачу в обробнику події.

Умови вимірювання

Щоб порівняння «до / після» у §5.5 мало сенс, обидва запуски мають бути зроблені однаково:

1. той самий URL — один і той самий деплой, не `file://` і не dev-сервер;
2. вікно в режимі інкогніто, розширення браузера вимкнені;
3. той самий preset throttling і той самий device mode;

4. **три запуски, у звіт — медіана**.

Розкид між запусками на незмінному коді часто перевищує ефект від реального виправлення. Якщо умови не зафіксовані, таблиця §5.5 не показує нічого — і це перше, що буде перевірено на захисті.

Експеримент

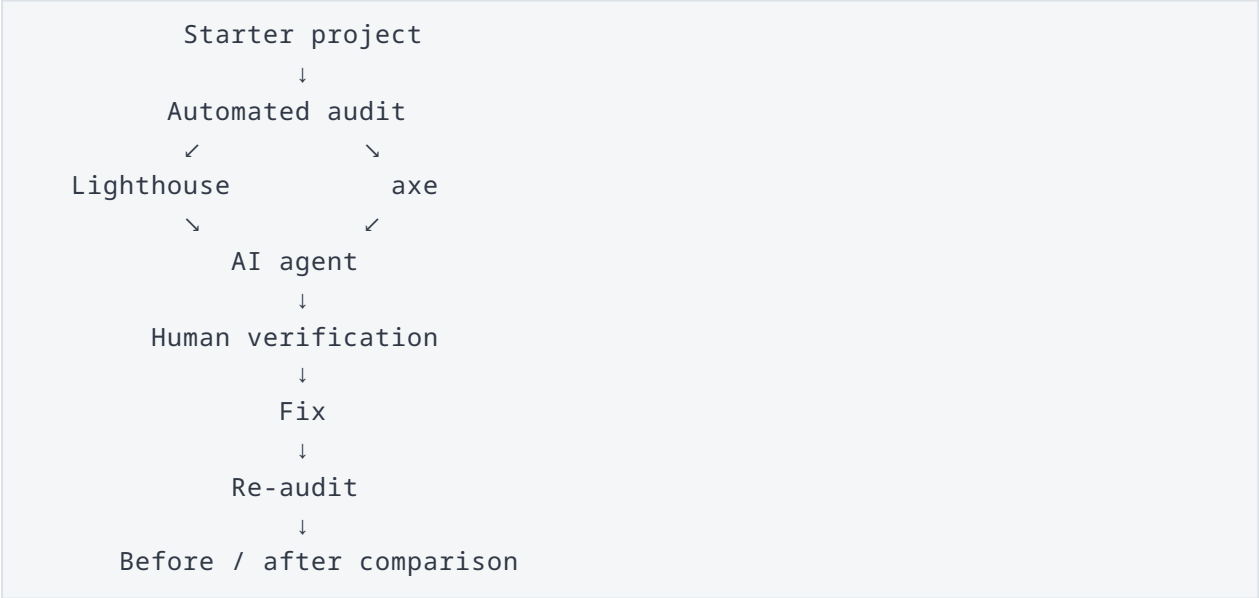
Запустіть Lighthouse Performance audit і дослідіть LCP, CLS, TBT, FCP, Speed Index, а також розділи Insights та Diagnostics.

Знайдіть щонайменше одну **причину** performance-проблеми та перевірте її експериментально: визначте, який саме елемент є LCP-елементом і що саме зсуває layout. Порівняйте результати до та після зміни.

5. Практична частина: AI-assisted Web Quality Audit

Використовуючи наданий starter-проєкт, проведіть комплексний аудит accessibility, технічного SEO та performance.

Starter-проєкт містить навмисно допущені проблеми різного типу. Завдання студента — не просто знайти їх за допомогою AI або автоматизованих інструментів, а **підтвердити** кожну проблему, визначити її причину, обґрунтувати спосіб виправлення та перевірити результат після внесення змін.



5.1. Початковий аудит

До внесення будь-яких змін проведіть первинну перевірку та зафіксуйте результати у звіті.

Accessibility. Semantic HTML і landmarks, heading hierarchy, `lang` документа, `<title>` сторінки (2.4.2), accessible names, links та buttons, keyboard navigation, видимий focus (2.4.7), images, forms і labels, повідомлення про помилки у формі (3.3.1), contrast, ARIA, Accessibility Tree.

Technical SEO. Тут перевірка розпадається на дві частини:

- *регресійна* — те, що вже було темою ЛР №3: `<title>`, meta description, heading structure, текст посилань, `canonical`;
- *новий шар* — Open Graph, JSON-LD, `robots.txt`, `sitemap.xml`, а також `<meta name="robots">` на сторінці.

Performance. LCP, CLS, TBT, решта Lighthouse diagnostics і — головне — причини проблем, а не лише числа. Виконуйте за умовами вимірювання з §4.7.

Канонічний origin

Робота публікується у двох місцях, тому сторінка буде доступна за двома різними URL, і `canonical` та `og:url` не можуть указувати на обидва одночасно. Оберіть **один** канонічний origin, використовуйте його в `canonical` та `og:url` на всіх сторінках і вмійте пояснити свій вибір. Перевірку `robots.txt` та `sitemap.xml` виконуйте на Vercel-деплої (див. §4.6).

5.2. AI-assisted audit

Передайте AI-агенту HTML, CSS, JavaScript, результати Lighthouse, результати accessibility scanner і, за необхідності, скріншоти.

AI не повинен одразу переписувати проєкт. Спочатку він повинен сформувати **audit report**, у якому для кожної виявленої проблеми зазначено:

Поле	Зміст
Category	Accessibility / SEO / Performance
Problem	Опис проблеми
Evidence	На чому ґрунтується висновок
Severity	Низька / середня / висока
Recommendation	Спосіб виправлення
Verification	Як перевірити результат

У звіті зафіксуйте, **який саме AI-агент і якої версії** використано та дату запуску: без цього результат неможливо відтворити.

5.3. Перевірка рекомендацій AI

Відберіть щонайменше **5 рекомендацій AI**. Для кожної визначте:

- чи підтверджується проблема;

- яким інструментом її можна перевірити;
- яким джерелом можна обґрунтувати рішення;
- чи буде рекомендація реалізована;
- якщо ні — чому її відхилено.

Особливу увагу приділіть рекомендаціям щодо ARIA, semantic HTML, accessible names, WCAG, structured data, `robots.txt` та performance.

5.4. Виправлення проблем

Реалізуйте обґрунтовані виправлення. Мінімально необхідно:

1. щонайменше **2 accessibility-проблеми**;
2. щонайменше **2 technical SEO-проблеми**;
3. щонайменше **1 performance-проблему** — або обґрунтовано показати, що зафіксована проблема не потребує зміни;
4. щонайменше **1 рекомендацію AI, яку ви відхилили**, з обґрунтуванням і посиланням на джерело.

Пункт 4 є обов'язковим результатом, а не побажанням: робота без жодної відхиленої рекомендації не демонструє головного вміння цієї лабораторної.

Основна структура та візуальна концепція starter-проєкту повинні бути збережені.

5.5. Повторний аудит

Після внесення змін повторно проведіть Lighthouse audit, accessibility scan, keyboard testing, перевірку Accessibility Tree, technical SEO check та performance audit — за тих самих умов, що й первинний (§4.7).

Перевірка	До	Після
Accessibility (ручна)	...	...
Lighthouse Accessibility	...	...
Automated (axe)	...	...
LCP	...	...
CLS	...	...
TBT	...	...
Performance diagnostics	...	...
Technical SEO	...	...

Приклад заповнення. Числа умовні й наведені лише для формату; ваші результати залежатимуть від деплою та пристрою. Над таблицею вкажіть умови вимірювання, а в клітинках — не лише число, а й коротку причину змін.

> Умови: Vercel-деплой, Chrome 140, інкогніто, Lighthouse → Mobile, preset «Slow 4G», медіана трьох запусків.

Перевірка	До	Після
Accessibility (ручна)	11 проблем, з них 5 не знайдено автоматично	2 проблеми залишилися (обґрунтування — у звіті)
Lighthouse Accessibility	71	100 — але це не означає відсутність проблем (див. рядок вище)
Automated (axe)	6 порушень (14 елементів), 1 «needs review»	0 порушень, 1 «needs review» перевірено вручну
LCP	6,4 с — LCP-елемент визначено, причину зафіксовано	2,2 с
CLS	0,27 — два джерела зсуву	0,02
TBT	0 мс — повільний обробник кліку TBT не бачить	0 мс; затримку взаємодії виміряно в Performance: 310 → 40 мс
Performance diagnostics	7 пунктів	2 пункти — свідомо не виправлено, з обґрунтуванням
Technical SEO	Lighthouse SEO 58; 6 проблем за ручною перевіркою	Lighthouse SEO 100; JSON-LD, robots.txt, sitemap.xml перевірено

Зробіть висновок, які проблеми були усунені, які залишилися, які виявилися хибними спрацюваннями та які потребують ручної перевірки.

> **Важливо!** Необхідно продемонструвати не лише позитивні результати. Уміння пояснити, чому рекомендація AI була відхилена або чому автоматичний інструмент не може остаточно визначити проблему, є частиною лабораторної роботи.

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-4-5: AI-агент для Web Quality Audit: accessibility, технічне SEO та Core Web Vitals
```

У листі вказати посилання на:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. результати дослідницьких завдань;
2. посилання на використані джерела;
3. результати початкового аудиту, **умови вимірювання** (§4.7) та обраний канонічний origin з обґрунтуванням (§5.1);
4. AI prompt, назву та версію використаного AI-агента, дату запуску;
5. результати AI-аудиту (audit report за таблицею §5.2);
6. аналіз щонайменше 5 рекомендацій AI, серед них щонайменше одна відхилена — з обґрунтуванням і посиланням на джерело;
7. опис внесених виправлень, а також обґрунтування проблем, які свідомо не виправлено;
8. скріншоти DevTools та автоматизованих перевірок;
9. порівняння результатів до / після (таблиця §5.5);
10. висновок про результати аудиту (§5.5).

> **Важливо!** Звіт не замінює результату: здається **і** звіт, **і** задеплоєний виправлений starter-проєкт. Без робочого деплою звіт не розглядається.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Навіть якщо аудит і частину виправлень запропонував AI-агент, ви повинні вміти пояснити кожну зміну, показати, як її перевірено, і відповісти на питання з демонстрацією екрана.

Орієнтовні питання:

Accessibility

- Відкрийте Accessibility Tree для конкретного елемента. Яку role та accessible name він має і звідки вони взялися?
- Перейдіть сторінкою лише клавіатурою. Чи можна дістатися всіх інтерактивних елементів? Чи видно фокус на кожному кроці?
- Яку проблему знайшов axe / Lighthouse? Покажіть її в HTML і назвіть критерій WCAG, якого вона стосується.
- Яку проблему ви знайшли **вручну**, а автоматизований інструмент — ні? Чому результат автоматичного аудиту не означає, що сторінка повністю accessible?

AI-аудит

- Яке виправлення AI ви прийняли і як перевірили, що воно коректне? Яку рекомендацію відхилили і чому?

Технічне SEO

- Який `canonical` ви обрали і чому саме цей URL?
- Покажіть JSON-LD. Чому обрано саме цей тип Schema.org і чи відповідає він контенту сторінки?
- Покажіть `robots.txt` і `sitemap.xml`. На якому origin вони справді працюють? Чим `robots.txt` відрізняється від `noindex`?

Core Web Vitals

- Який елемент є LCP-елементом? Яке значення LCP отримано до та після оптимізації і за яких умов ви міряли?
- Яка причина CLS на сторінці, якщо він виникає?
- Чому INP немає у звичайному звіті Lighthouse? Покажіть у DevTools → Performance, як ви знайшли причину затримки взаємодії.

8. Оцінювання

1. Успішний захист — **4 бали**.
2. Додаткові **2 бали** за однієї з умов:
   - термін здачі до **30.09.2026**, АБО
   - робота на парі.